T14 Grepai : Recherche Sémantique

Technique Intermediate

Trouver du code par intention, pas par regex

Principe : chercher par intention

Grep natif (ripgrep) est idéal quand on connaît le texte exact à trouver. Grepai prend le relai quand on cherche « le code qui gère X » sans connaître les noms exacts des fonctions ou des variables. Il s'appuie sur des embeddings locaux via Ollama et le modèle `nomic-embed-text`, ce qui maintient cette étape d'indexation en local. Les résultats transmis à Claude peuvent néanmoins entrer dans le contexte du fournisseur configuré.

Mesurer la valeur : comparer précision de recherche, taille du contexte retourné et coût par tâche acceptée sur des recherches représentatives.

Les 5 outils principaux

`grepai_search` : recherche par description naturelle. Les requêtes courtes (1-3 mots-clés) donnent de meilleurs résultats que des phrases longues.

```
grepai_search("payment flow")
grepai_search("JWT validation")
grepai_search("rate limiting middleware")
```

`grepai_trace_callers` : trouve toutes les fonctions qui appellent un symbole donné.

`grepai_trace_callees` : liste ce qu'une fonction appelle en interne.

`grepai_trace_graph` : génère le graphe de dépendances complet autour d'un symbole.

`grepai_index_status` : vérifie que l'index est à jour avant de lancer des recherches.

Graphe d'appels en pratique

Le cas d'usage le plus puissant de grepai est l'analyse de dépendances. Avant de refactorer une fonction, savoir qui l'appelle évite les régressions silencieuses.

```
# Qui appelle validateJWT ?
grepai_trace_callers("validateJWT")
→ ApiGateway, AdminPanel, UserController

# Que fait validateJWT en interne ?
grepai_trace_callees("validateJWT")
→ decodeToken, checkExpiry, verifySignature
```

Profondeur recommandée : 2 par défaut, 3 au maximum pour les analyses complexes.

Grepai vs Grep natif

Situation	Outil recommandé
Pattern exact connu (`"authenticate"`)	Grep natif
Recherche par concept ou intention	Grepai
Analyse de graphe d'appels	Grepai (`trace_callers`)
Refactoring multi-fichiers	Grepai + Serena
Regex avancée sur structure AST	ast-grep

Règle simple : si on peut écrire la regex, utiliser Grep. Sinon, grepai.

Setup

```
# Installer grepai
curl -sSL https://raw.githubusercontent.com/yoanbernabeu/grepai/main/install.sh | sh

# Initialiser dans le projet (choisir: ollama, nomic-embed-text)
grepai init

# Vérifier le statut
grepai status
```

Prérequis : Ollama installé avec le modèle `nomic-embed-text` disponible. En cas d'erreur « connection refused » : `brew services start ollama`.

Bonnes pratiques

Toujours appeler `grepai_index_status` en début de session si le codebase a évolué depuis la dernière indexation. Limiter les résultats à 5 par défaut et augmenter uniquement si les premiers résultats ne sont pas pertinents. Grepai découvre les fichiers, Grep natif affine ensuite avec une regex précise : les deux outils se complètent naturellement.